

Floe: A Change-Data-Capture Lakehouse on Apache Iceberg, Spark and Trino

Aliyan Zulfiqar (Group Leader) Sehrosh Riaz
SE-315 Cloud Computing, Semester Project (Group 04)

Department of Computer Science, NBC

Instructor: Dr. Mohsin Raza Jafri Lab Engineer: Engr. Muhammad Abid Hussain

Repository: <https://github.com/zorhehs/floe>

Abstract—Running analytics on an operational database competes with its transactions, and nightly batch exports deliver stale data. We present Floe, a lakehouse that continuously replicates row-level changes from PostgreSQL into Apache Iceberg tables and serves them to SQL analytics through Trino. Debezium reads the PostgreSQL write-ahead log and publishes change events to Kafka; a Spark Structured Streaming job applies each micro-batch with an idempotent `MERGE`, evolving table schemas automatically. Airflow orchestrates Spark maintenance jobs for partition evolution, sorted compaction, snapshot expiry and GDPR-style erasure, and Prometheus and Grafana monitor freshness and file health. On TPC-H scale factor 1 (1,650,030 rows) the full load became queryable at 4,844 rows/s end to end, individual changes reached Trino with a median latency of 7.3s, and source and lakehouse matched exactly by row-level checksums. Re-laying out the streamed tables removed all 105 merge-on-read delete files, reduced stored objects by 89% and made a suite of eight analytical queries $3.25\times$ faster overall.

Index Terms—lakehouse, change data capture, Apache Iceberg, Spark Structured Streaming, Debezium, Trino, compaction

I. INTRODUCTION AND MOTIVATION

Organisations increasingly want analytics over the same data their applications are modifying: orders, balances and inventory. Running heavy analytical SQL directly on the operational database competes with transaction processing, while the traditional alternative of exporting full tables to a warehouse every night is costly, because each run copies all data, and stale, because results can be a day old. A lakehouse combines low-cost open storage with the transactional table semantics of a warehouse [1], and change data capture (CDC) feeds it incrementally by reading only the changes recorded in the database log.

This project builds such a system, which we call *Floe*, entirely from open-source components and measures its behaviour. The objectives were to (i) stream inserts, updates and deletes from PostgreSQL into Apache Iceberg; (ii) keep the copy correct and fresh, including across source schema changes; (iii) demonstrate schema evolution, time travel and row-level deletion; (iv) counter the small-file problem caused by streaming writes with compaction and partition optimisation orchestrated by Airflow, and quantify query performance and storage before and after; and (v) make the system observable, reproducible and tested.

II. BACKGROUND

Change data capture. PostgreSQL records every committed change in its write-ahead log (WAL). With `wal_level=logical`, a logical replication slot decodes the WAL into row changes [4]. Debezium consumes the slot through the built-in `pgoutput` plugin, first snapshots existing rows and then emits one event per changed row containing the new row image, the operation, the commit time and the log sequence number (LSN) [3]. Because the slot retains WAL until the consumer confirms it, changes are not lost while the consumer is unavailable.

Apache Iceberg is an open table format in which a table is a tree of immutable files: Parquet data files, delete files, manifests and a metadata file [2]. A catalog stores an atomic pointer to the current metadata file, and every commit produces a new snapshot. This provides ACID transactions with optimistic concurrency, time travel, schema evolution by column identifier, and hidden partitioning whose specification can be changed without rewriting existing data. Format version 2 adds row-level deletes: with *merge-on-read*, an update writes the new row and a small position-delete file instead of rewriting the affected data file.

Engines. Spark Structured Streaming processes Kafka in micro-batches with checkpointed offsets [5]; Trino is a distributed SQL engine that reads Iceberg tables directly from object storage [6].

III. SYSTEM ARCHITECTURE

Fig. 1 shows the components and data flow. All services run as containers defined in a single Docker Compose file. Table I summarises the components and the reasons for choosing them.

IV. IMPLEMENTATION

A. Change Capture

An initialisation script creates separate database roles for the application, for Debezium (with the `REPLICATION` attribute) and for the catalog, together with the TPC-H shaped schema shop. A publication `FOR TABLES IN SCHEMA shop` includes tables created later. The connector is registered idempotently and uses `decimal.handling.mode=string` with `column.propagate.source.type`, so that a `numeric(12,2)` column arrives as an exact value

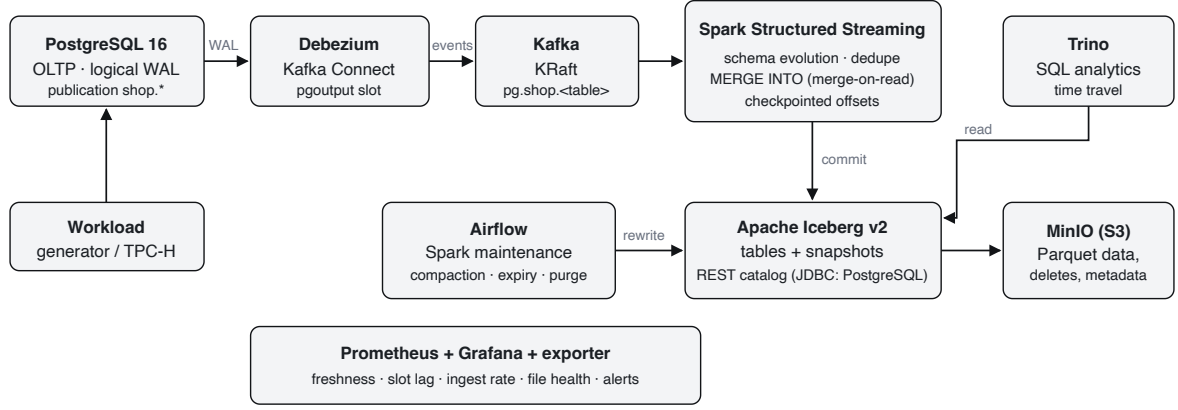


Fig. 1. Floe components and data flow.

TABLE I
COMPONENTS AND DESIGN CHOICES

Layer	Component	Reason
Source	PostgreSQL 16	Logical decoding; publication covers future tables
CDC	Debezium 2.7	Snapshot plus streaming; exact decimals with source precision
Transport	Kafka 3.9 (KRaft)	Durable, replayable log ordered per key
Processing	Spark 3.5	foreachBatch gives full SQL (MERGE) per batch
Table format	Iceberg 1.9 (v2)	Engine-neutral; merge-on-read; partition evolution
Catalog	Iceberg REST	Standard API for Spark and Trino; state in PostgreSQL
Storage	MinIO	S3 API; replaceable by S3 or Azure Data Lake Storage
Query	Trino 476	Interactive SQL, metadata tables, time travel
Orchestration	Airflow 2.10	Scheduling, retries and history of maintenance jobs
Monitoring	Prometheus, Grafana	Pipeline-specific exporter and alert rules

together with its precision and scale. The setting `max_slot_wal_keep_size` bounds the WAL that a stopped consumer can retain.

B. Streaming Ingestion

The Spark job subscribes to all topics of the schema and processes each micro-batch per table:

- 1) *Schema discovery*. Each Debezium event carries its schema. The job collects the distinct schemas in the batch by cutting the message text before the payload, which avoids parsing every event, and maps Debezium logical types (dates, timestamps, decimals, binary) to Iceberg types.
- 2) *Schema evolution*. Unknown tables are created as format-v2, merge-on-read tables. New source columns are added with `ALTER TABLE ... ADD COLUMN`; promotions that Iceberg supports in place (integer to big-

int, float to double, wider decimal) are applied with `ALTER COLUMN ... TYPE`; other changes are cast and logged.

- 3) *Deduplication*. Only the latest event per primary key, by Kafka offset, is kept.

- 4) *Merge*. A single statement applies the batch:

```
MERGE INTO lakehouse.shop.orders t USING batch s
ON t.o_orderkey = s.o_orderkey
WHEN MATCHED AND s._cdc_op = 'd' THEN DELETE
WHEN MATCHED THEN UPDATE SET ...
WHEN NOT MATCHED AND s._cdc_op <> 'd' THEN INSERT ...
```

Because the statement is idempotent, replaying the last micro-batch after a failure yields the same table state, which together with checkpointed offsets gives effectively-once results. Commits that conflict with a concurrent maintenance job are retried with exponential backoff. Each row also stores the operation, the source commit time, the LSN and the ingestion time, which are used to measure freshness and to audit changes.

C. Maintenance and Orchestration

A Spark job exposes the Iceberg maintenance procedures as composable steps, and three Airflow DAGs schedule them: hourly housekeeping, a manual layout optimisation, and a GDPR purge. The layout optimisation partitions orders by months(`o_orderdate`), sorts it by order date and customer, sorts customer by nation and key, and then runs `rewrite_data_files` with the sort strategy, `rewrite_position_delete_files`, `rewrite_manifests`, `expire_snapshots` and `remove_orphan_files`. Partition evolution changes only metadata, so the streaming job continues without restarting. Orphan removal receives an S3 listing through `file_list_view` and only deletes files older than 24 h, which protects files of in-flight commits.

D. Monitoring, Testing and Reproducibility

A Prometheus exporter writes a heartbeat row into PostgreSQL every 15 s and reports the age of the newest heartbeat visible through Trino, which is the end-to-end freshness. It

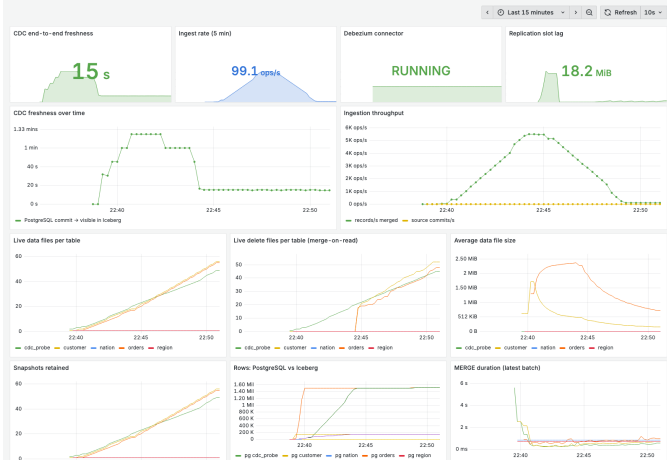


Fig. 2. Grafana dashboard during the final run: the initial-load backlog drains, then freshness stays steady under the live workload while data and delete files accumulate.

also reports replication-slot lag, connector state, ingest rate, merge duration and the number of data and delete files per table (Fig. 2). Alert rules cover stale data, a stopped connector, WAL growth, small-file accumulation and a stalled stream. Correctness is checked by comparing row counts and an order-independent `checksum()` over all columns between the PostgreSQL and Iceberg catalogs in Trino. The repository contains unit tests, a full-stack integration test, a GitHub Actions workflow and scripts for setup, demonstration and clean-up.

V. EVALUATION

A. Setup

All services ran on one laptop (Apple silicon, 8 cores) with Docker limited to 8GB of memory and Spark in local mode, so the results describe a single node. The data set is TPC-H scale factor 1 (150 000 customers and 1 500 000 orders) generated by Trino’s `tpch` connector [9], followed by a synthetic OLTP workload of 100 operations per second for five minutes (45% new orders, 20% order updates, 10% order deletes, 15% balance updates, 5% new and 5% deleted customers). Spark used a 10 s trigger and at most 100 000 records per batch. Query times are medians of 5 runs after one warm-up run.

B. Ingestion Throughput

Loading 1,650,030 rows into PostgreSQL took 27 s, and all rows were queryable in Iceberg with matching checksums 341 s after the load started, i.e. 4,844 rows/s end to end. Over 62 micro-batches (1,688,434 change events) the job processed 3,575 events/s while busy, with a peak of 7,752 events/s and a median batch time of 3.7 s (95th percentile 20.5 s). The MERGE into `orders` took 0.9 s at the median (1.9 s at the 95th percentile) even for batches of about 90 000 rows; most of the batch time is spent decoding the self-describing JSON events on a single executor (Fig. 3).

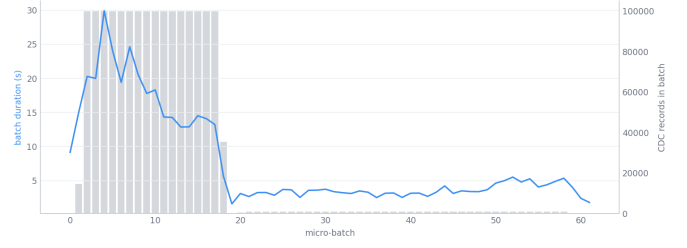


Fig. 3. Change events (bars) and processing time (line) per micro-batch: initial load, then live workload.

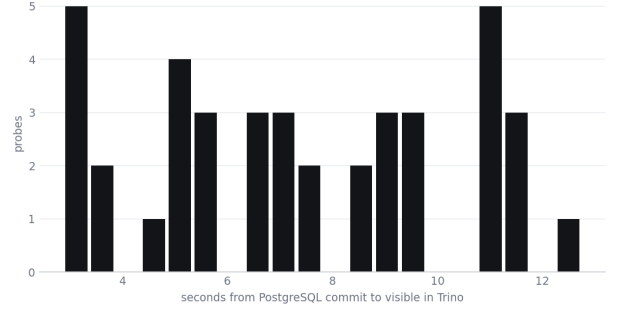


Fig. 4. Distribution of end-to-end freshness (PostgreSQL commit to visible in Trino).

C. End-to-End Freshness

During the workload, probe rows were inserted into PostgreSQL every 2 s and polled through Trino. All 40 probes became visible (0 timed out); the time from commit to visibility had a median of 7.3 s, a 95th percentile of 11.7 s and a maximum of 12.7 s (Fig. 4). Freshness is bounded below by the trigger interval plus the merge and commit time; a shorter trigger reduces latency at the cost of more and smaller files.

D. Layout Optimisation: Files, Storage and Queries

After the load and workload, `orders` consisted of 57 data files averaging 717 KB plus 49 position-delete files that every query must merge at read time. The optimisation DAG rewrote it into 80 sorted files (average 581 KB) in monthly partitions and expired the superseded snapshots (Table II, Fig. 5). The number of objects in storage fell by 89% (from 2,151 to 246), while total bytes changed by only -0.4% (58.5 MB to 58.3 MB): five minutes of workload produce small delete files, so the gain is in object count, which drives listing and planning cost on object storage, rather than in bytes.

Table III compares the query suite. Overall the suite ran $3.25\times$ faster, with the largest gain for the point lookup query ($8.8\times$). Date-filtered queries benefit from partition pruning; the single-month query read 19,488 rows instead of 1,513,192. Every query benefits from reading fewer files without delete files to merge. Queries that did not improve: full aggregate ($0.95\times$). A whole-table scan gains nothing from pruning and now opens more, smaller monthly files.

Partition granularity. An ablation on an already compacted table compared one compacted file with the same data in

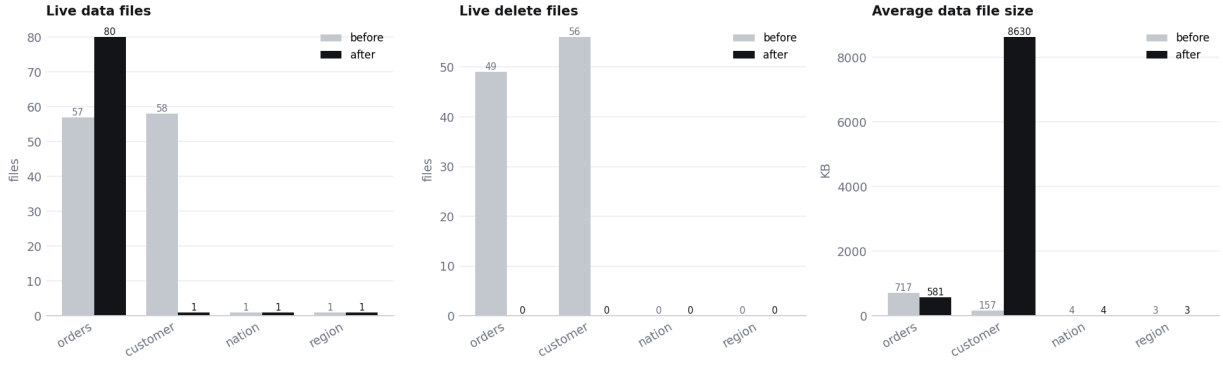


Fig. 5. Live data files, delete files and average data-file size per table before (grey) and after (black) optimisation.

TABLE II
LIVE FILES BEFORE → AFTER OPTIMISATION

Table	Data files	Delete files	Avg. file (KB)	Snapshots
orders	57 → 80	49 → 0	717 → 581	57 → 1
customer	58 → 1	56 → 0	157 → 8,630	58 → 1

TABLE III
TRINO QUERY LATENCY (MEDIAN, MS) AND ROWS READ (MILLIONS)

#	Query	Before	After	Speed-up	Read bef.	Read aft.
Q01	point lookup	971	110	8.81×	1.23	1.51
Q02	single month	340	46	7.38×	1.51	0.02
Q03	quarterly revenue	1,287	174	7.39×	1.51	0.69
Q04	revenue by nation	8,089	1,381	5.86×	1.81	0.53
Q05	top customers	8,934	3,646	2.45×	1.81	1.81
Q06	full aggregate	950	1,004	0.95×	1.51	1.51
Q07	segment priority	750	178	4.22×	1.69	0.23
Q08	customer history	240	91	2.64×	1.49	1.51
total		21,561	6,630	3.25×		

monthly partitions (80 files of about 570 KB). Partitioning changed the one-month query latency by -18%, but slowed the full-table queries by +21% and +35%. At scale factor 1 a month holds only about 20 000 orders; monthly partitions pay off as volume grows, while at small scale a coarser transform or a sort order alone is preferable. Partition evolution makes this a metadata-only decision.

E. Compaction Overhead

The complete optimisation DAG took 170s of wall time while the streaming job kept running (Table IV). Most of the time is Spark start-up and the per-table orphan scan; the sorted rewrite of `orders` itself took well under a minute.

F. Fault Tolerance and Correctness

During an earlier freshness measurement the Docker virtual machine ran out of memory and the operating system killed the Spark driver. The container restarted, resumed from the checkpointed offsets and re-applied the interrupted batch; because the merge is idempotent no change was lost or duplicated. Median freshness in that run remained 10.8s, but

TABLE IV
OPTIMISATION DAG TASK DURATIONS

Airflow task	Duration (s)
apply_layout_and_compact	43.9
rewrite_deletes_and_manifests	22.8
expire_snapshots	23.1
remove_orphan_files	67.7

TABLE V
POSTGRESQL VERSUS ICEBERG (ROW-LEVEL CHECKSUM)

Table	Rows	Before / after optimisation
customer	149,932	match / match
nation	25	match / match
orders	1,513,192	match / match
region	5	match / match

probes caught in the restart took up to 78.6s. Memory limits were reduced afterwards. Row counts and checksums of every table matched between PostgreSQL and Iceberg before and after optimisation (Table V), and the same check passed after the initial load, after the workload, after replaying the Kafka topics into an empty lake, and in the integration test.

G. Functional Demonstrations

Adding a column and widening an integer column to bigint in PostgreSQL reached Iceberg within about 10s as metadata-only changes. Time-travel queries by snapshot identifier and by timestamp returned the earlier values of an updated order. A customer deleted in PostgreSQL disappeared from current queries within about 10s but remained readable through time travel until the purge DAG rewrote the affected files and expired the history.

VI. TECHNICAL CHALLENGES AND LESSONS LEARNED

Several problems appeared only under realistic load. Concurrent commits made the REST catalog's SQLite store return `SQLITE_BUSY`; moving catalog state to PostgreSQL solved it. Spark `MERGE` failed on Iceberg tables with identifier fields, so the primary key is stored as a table property instead.

`remove_orphan_files` requires a Hadoop file system for S3, which the stack does not include, so the file listing is supplied through `file_list_view`. Default memory sizing of Trino, Kafka Connect and the maintenance job caused failures during the 1.5-million-row load and rewrite; explicit heap limits, a byte-bounded Debezium queue and spill-to-disk fixed them. Finally, the sum of container memory exceeded the 8 GB virtual machine, which led to the driver restart described above. The main lesson is that the operational aspects of CDC ingestion (concurrent writers, metadata growth, small files and observability) matter as much as the data path, and that a repeatable correctness check makes each of them easier to resolve.

VII. CONCLUSION AND FUTURE WORK

Floe makes changes from PostgreSQL queryable in Iceberg within seconds, keeps an exact copy of the source, evolves schemas without intervention, supports time travel and verifiable erasure, and uses scheduled optimisation to turn streaming-induced small files into a layout that made the query suite $3.25\times$ faster while reducing stored objects by 89%. The evaluation is limited to a single node, freshness is bounded by the micro-batch trigger, and dropped source columns remain in Iceberg. Future work includes a comparison with Apache Hudi on the same workload, column-level lineage with Apache Atlas, Avro with a schema registry to reduce serialisation cost, a Kubernetes deployment on Azure with Data Lake Storage and Event Hubs, and Apache Flink for lower-latency ingestion.

REFERENCES

- [1] M. Armbrust, A. Ghodsi, R. Xin, and M. Zaharia, “Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics,” in *Proc. CIDR*, 2021.
- [2] Apache Software Foundation, “Apache Iceberg table specification and Spark procedures,” <https://iceberg.apache.org/docs/latest/>.
- [3] Debezium, “Debezium connector for PostgreSQL,” version 2.7, <https://debezium.io/documentation/>.
- [4] PostgreSQL Global Development Group, “Logical replication,” PostgreSQL 16 documentation.
- [5] Apache Software Foundation, “Structured Streaming programming guide,” Spark 3.5, <https://spark.apache.org/docs/3.5.6/>.
- [6] Trino Software Foundation, “Iceberg connector,” <https://trino.io/docs/current/connector/iceberg.html>.
- [7] Apache Software Foundation, “Apache Kafka documentation,” version 3.9.
- [8] Apache Software Foundation, “Apache Airflow documentation,” version 2.10.
- [9] Transaction Processing Performance Council, “TPC Benchmark H standard specification.”

APPENDIX A REPRODUCING THE RESULTS

```
git clone https://github.com/zorhehs/floe && cd floe
./scripts/setup.sh # checks, .env, jars, images
./scripts/run_demo.sh --full # stack, TPC-H sfl, workload,
                          # validation, demos, benchmark
# or step by step
make up && make load && make bench && make validate
python3 docs/report/latex/build_tex.py # rebuild this paper
```

APPENDIX B KEY CONFIGURATION

```
# Debezium connector (config/debezium/postgres-cdc.json,
excerpt)
plugin.name=pgoutput publication.name=dbz_publication
slot.name=debezium_shop topic.prefix=pg snapshot.mode=
initial
decimal.handling.mode=string column.propagate.source.type=
shop\.*
max.queue.size.in.bytes=134217728 heartbeat.interval.ms
=10000

# Iceberg table properties set by the streaming job
format-version=2 write.{delete,update,merge}.mode=merge-on-
read
write.parquet.compression-codec=zstd

# Layout optimisation (src/topologies/spark/maintenance.py)
ALTER TABLE lakehouse.shop.orders ADD PARTITION FIELD
months(o_orderdate);
ALTER TABLE lakehouse.shop.orders WRITE ORDERED BY
o_orderdate, o_custkey;
CALL lakehouse.system.rewrite_data_files(table => 'shop.
orders',
strategy => 'sort', options => map('rewrite-all','true',
'target-file-size-bytes','134217728','partial-progress.
enabled','true'));
```